Chapter 1: Introduction to Operating Systems

An operating system is system software that manages computer hardware and software resources and provides common services for computer programs. The kernel is the core component of an operating system and runs in a privileged mode called kernel mode. User applications run in user mode and request services from the kernel through system calls. Early batch systems executed jobs one after another without user interaction. Time-sharing systems later allowed many users to interact with a computer at the same time by rapidly switching the processor between them. A real-time operating system guarantees that tasks are completed within strict timing constraints, which is essential in embedded and safety-critical systems.An operating system is system software that manages computer hardware and software resources and provides common services for computer programs. The kernel is the core component of an operating system and runs in a privileged mode called kernel mode. User applications run in user mode and request services from the kernel through system calls. Early batch systems executed jobs one after another without user interaction. Time-sharing systems later allowed many users to interact with a computer at the same time by rapidly switching the processor between them. A real-time operating system guarantees that tasks are completed within strict timing constraints, which is essential in embedded and safety-critical systems.

Chapter 1: Introduction to Operating Systems (continued)

An operating system is system software that manages computer hardware and software resources and provides common services for computer programs. The kernel is the core component of an operating system and runs in a privileged mode called kernel mode. User applications run in user mode and request services from the kernel through system calls. Early batch systems executed jobs one after another without user interaction. Time-sharing systems later allowed many users to interact with a computer at the same time by rapidly switching the processor between them. A real-time operating system guarantees that tasks are completed within strict timing constraints, which is essential in embedded and safety-critical systems.An operating system is system software that manages computer hardware and software resources and provides common services for computer programs. The kernel is the core component of an operating system and runs in a privileged mode called kernel mode. User applications run in user mode and request services from the kernel through system calls. Early batch systems executed jobs one after another without user interaction. Time-sharing systems later allowed many users to interact with a computer at the same time by rapidly switching the processor between them. A real-time operating system guarantees that tasks are completed within strict timing constraints, which is essential in embedded and safety-critical systems.

Chapter 2: Process Management

A process is a program in execution. Each process has a process control block that stores its state, program counter, registers, and scheduling information. A process can be in the new, ready, running, waiting, or terminated state. The scheduler decides which ready process receives the CPU next. First-come first-served scheduling runs processes in order of arrival, while round-robin scheduling gives each process a fixed time quantum. Shortest job first scheduling minimizes average waiting time when burst lengths are known. A context switch saves the state of the running process and restores the state of the next one; context switching is pure overhead because no useful work is done during it. Threads are lightweight units of execution that share the address space of their parent process.A process is a program in execution. Each process has a process control block that stores its state, program counter, registers, and scheduling information. A process can be in the new, ready, running, waiting, or terminated state. The scheduler decides which ready process receives the CPU next. First-come first-served scheduling runs processes in order of arrival, while round-robin scheduling gives each process a fixed time quantum. Shortest job first scheduling minimizes average waiting time when burst lengths are known. A context switch saves the state of the running process and restores the state of the next one; context switching is pure overhead because no useful work is done during it. Threads are lightweight units of execution that share the address space of their parent process.

Chapter 2: Process Management (continued)

A process is a program in execution. Each process has a process control block that stores its state, program counter, registers, and scheduling information. A process can be in the new, ready, running, waiting, or terminated state. The scheduler decides which ready process receives the CPU next. First-come first-served scheduling runs processes in order of arrival, while round-robin scheduling gives each process a fixed time quantum. Shortest job first scheduling minimizes average waiting time when burst lengths are known. A context switch saves the state of the running process and restores the state of the next one; context switching is pure overhead because no useful work is done during it. Threads are lightweight units of execution that share the address space of their parent process.A process is a program in execution. Each process has a process control block that stores its state, program counter, registers, and scheduling information. A process can be in the new, ready, running, waiting, or terminated state. The scheduler decides which ready process receives the CPU next. First-come first-served scheduling runs processes in order of arrival, while round-robin scheduling gives each process a fixed time quantum. Shortest job first scheduling minimizes average waiting time when burst lengths are known. A context switch saves the state of the running process and restores the state of the next one; context switching is pure overhead because no useful work is done during it. Threads are lightweight units of execution that share the address space of their parent process.

Chapter 3: Memory Management

Memory management keeps track of every byte of memory and decides which process gets memory and when. Paging divides physical memory into fixed-size frames and logical memory into pages of the same size, which eliminates external fragmentation. A page table maps page numbers to frame numbers. The translation lookaside buffer is a small hardware cache that stores recent page table entries to speed up address translation. Virtual memory allows a process to run even when only part of it is loaded in physical memory. When a referenced page is not in memory, a page fault occurs and the operating system loads the page from disk. Thrashing happens when a system spends more time swapping pages than executing processes. The least recently used page replacement algorithm evicts the page that has not been used for the longest time.Memory management keeps track of every byte of memory and decides which process gets memory and when. Paging divides physical memory into fixed-size frames and logical memory into pages of the same size, which eliminates external fragmentation. A page table maps page numbers to frame numbers. The translation lookaside buffer is a small hardware cache that stores recent page table entries to speed up address translation. Virtual memory allows a process to run even when only part of it is loaded in physical memory. When a referenced page is not in memory, a page fault occurs and the operating system loads the page from disk. Thrashing happens when a system spends more time swapping pages than executing processes. The least recently used page replacement algorithm evicts the page that has not been used for the longest time.

Chapter 3: Memory Management (continued)

Memory management keeps track of every byte of memory and decides which process gets memory and when. Paging divides physical memory into fixed-size frames and logical memory into pages of the same size, which eliminates external fragmentation. A page table maps page numbers to frame numbers. The translation lookaside buffer is a small hardware cache that stores recent page table entries to speed up address translation. Virtual memory allows a process to run even when only part of it is loaded in physical memory. When a referenced page is not in memory, a page fault occurs and the operating system loads the page from disk. Thrashing happens when a system spends more time swapping pages than executing processes. The least recently used page replacement algorithm evicts the page that has not been used for the longest time.Memory management keeps track of every byte of memory and decides which process gets memory and when. Paging divides physical memory into fixed-size frames and logical memory into pages of the same size, which eliminates external fragmentation. A page table maps page numbers to frame numbers. The translation lookaside buffer is a small hardware cache that stores recent page table entries to speed up address translation. Virtual memory allows a process to run even when only part of it is loaded in physical memory. When a referenced page is not in memory, a page fault occurs and the operating system loads the page from disk. Thrashing happens when a system spends more time swapping pages than executing processes. The least recently used page replacement algorithm evicts the page that has not been used for the longest time.

Chapter 4: File Systems

A file system controls how data is stored and retrieved on a storage device. Files are organized into directories, which can form a tree structure. An inode stores metadata about a file, such as its size, owner, permissions, and the locations of its data blocks, but not its name. Journaling file systems record intended changes in a log before applying them, which allows faster recovery after a crash. Contiguous allocation stores a file in consecutive blocks and gives fast sequential access but suffers from external fragmentation. Linked allocation stores a pointer to the next block inside each block. Indexed allocation uses an index block that holds pointers to all data blocks of the file.A file system controls how data is stored and retrieved on a storage device. Files are organized into directories, which can form a tree structure. An inode stores metadata about a file, such as its size, owner, permissions, and the locations of its data blocks, but not its name. Journaling file systems record intended changes in a log before applying them, which allows faster recovery after a crash. Contiguous allocation stores a file in consecutive blocks and gives fast sequential access but suffers from external fragmentation. Linked allocation stores a pointer to the next block inside each block. Indexed allocation uses an index block that holds pointers to all data blocks of the file.

Chapter 4: File Systems (continued)

A file system controls how data is stored and retrieved on a storage device. Files are organized into directories, which can form a tree structure. An inode stores metadata about a file, such as its size, owner, permissions, and the locations of its data blocks, but not its name. Journaling file systems record intended changes in a log before applying them, which allows faster recovery after a crash. Contiguous allocation stores a file in consecutive blocks and gives fast sequential access but suffers from external fragmentation. Linked allocation stores a pointer to the next block inside each block. Indexed allocation uses an index block that holds pointers to all data blocks of the file.A file system controls how data is stored and retrieved on a storage device. Files are organized into directories, which can form a tree structure. An inode stores metadata about a file, such as its size, owner, permissions, and the locations of its data blocks, but not its name. Journaling file systems record intended changes in a log before applying them, which allows faster recovery after a crash. Contiguous allocation stores a file in consecutive blocks and gives fast sequential access but suffers from external fragmentation. Linked allocation stores a pointer to the next block inside each block. Indexed allocation uses an index block that holds pointers to all data blocks of the file.